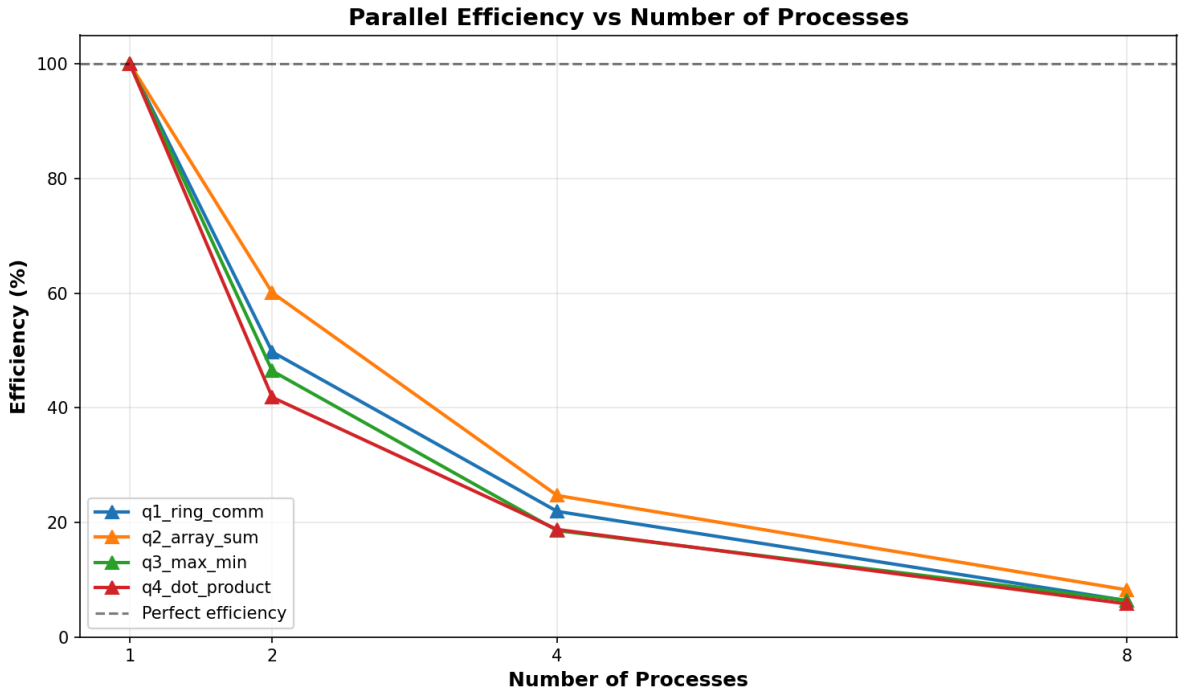
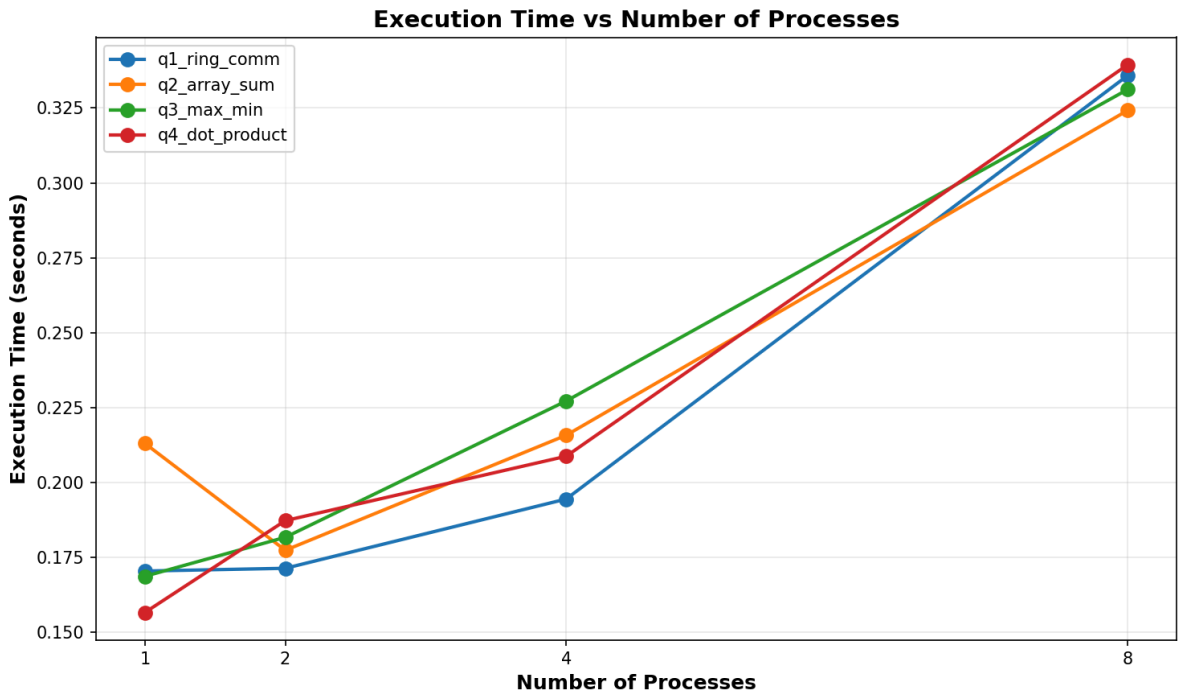
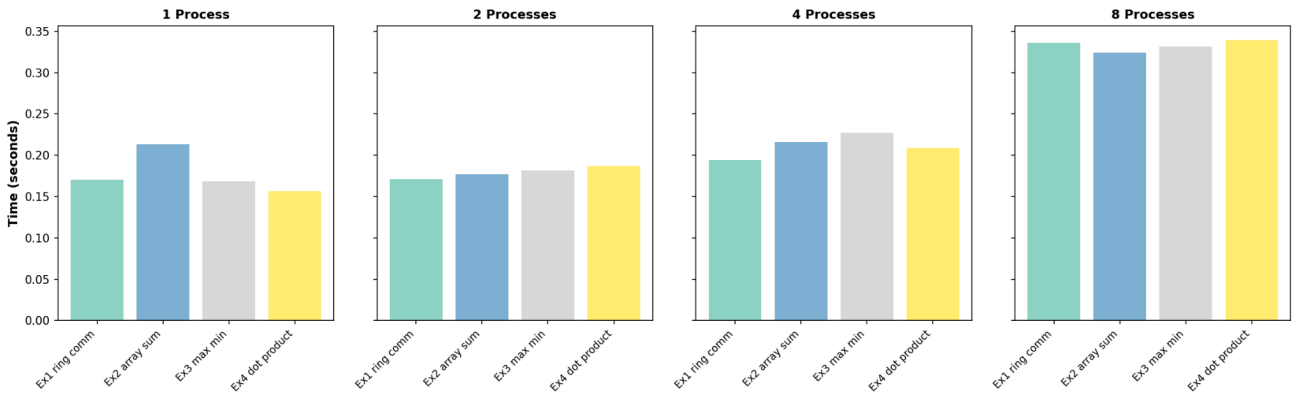
Assignment 4- UCS645

Aarav Dudeja








```
Running baseline tests (1 process)...
  q1_ring_comm ... done (.170366141s)
  q2_array_sum ... done (.212987796s)
  q3_max_min ... done (.168611964s)
  q4_dot_product ... done (.156534976s)
```

Q1)

Number of processors	TIme	Speed up	Efficiency
1	0.170	1.0	100%
2	0.171	0.99	49.7%
4	0.194	0.88	21.9%
8	0.336	0.51	6.3%

Speedup stays below 1 after 2+ processes. The overhead is taking most of the time. Increasing processors adds message startup/syncronization overhead and hence the runtime increases.

```
Running multi-process scaling tests...
  Testing q1_ring_comm...
    np=2 ... done (.171284854s, speedup .994x)
    np=4 ... done (.194409958s, speedup .876x)
    np=8 ... done (.335889049s, speedup .507x)
```

Q2) The output for all the values is 5050. The best performance occurs at 2 processes after which overhead dominates and reduce performance

Number of	TIme	Speed up	Efficiency

processors			
1	0.213	1.0	100%
2	0.177	1.20	60.06%
4	0.216	0.99	24.6%
8	0.324	0.66	8.2%

```
Testing q2_array_sum...
  np=2 ... done (.177304270s, speedup 1.201x)
  np=4 ... done (.215710450s, speedup .987x)
  np=8 ... done (.324231441s, speedup .656x)
```

Q3) Functional behaviour is correct and values change every run. Problem size per process is tiny (10 random numbers), so collectives and startup dominate. It does not scale for performance at this size.

Number of processors	TIme	Speed up	Efficiency
1	0.169	1.0	100%
2	0.182	0.93	46.4%
4	0.227	0.74	18.5%
8	0.331	0.51	6.3%

```
Testing q3_max_min...
  np=2 ... done (.181640468s, speedup .928x)
  np=4 ... done (.227147931s, speedup .742x)
  np=8 ... done (.331216115s, speedup .509x)
```

Q4) Vector length is only 8, so communication and process launch overhead are much larger than compute. Parallelization is correct logically, but not beneficial for such a small dot product workload

Number of processors	TIme	Speed up	Efficiency

1	0.157	1.0	100%
2	0.187	0.84	41.4%
4	0.209	0.75	18.7%
8	0.339	0.46	5.76%

```
Testing q4_dot_product...
  np=2 ... done (.187207538s, speedup .836x)
  np=4 ... done (.208698180s, speedup .750x)
  np=8 ... done (.339488202s, speedup .461x)
```